# LinkedHashMap and TreeMap

8 min read  Updated 28 Jul 2026  Topic 19 of 65

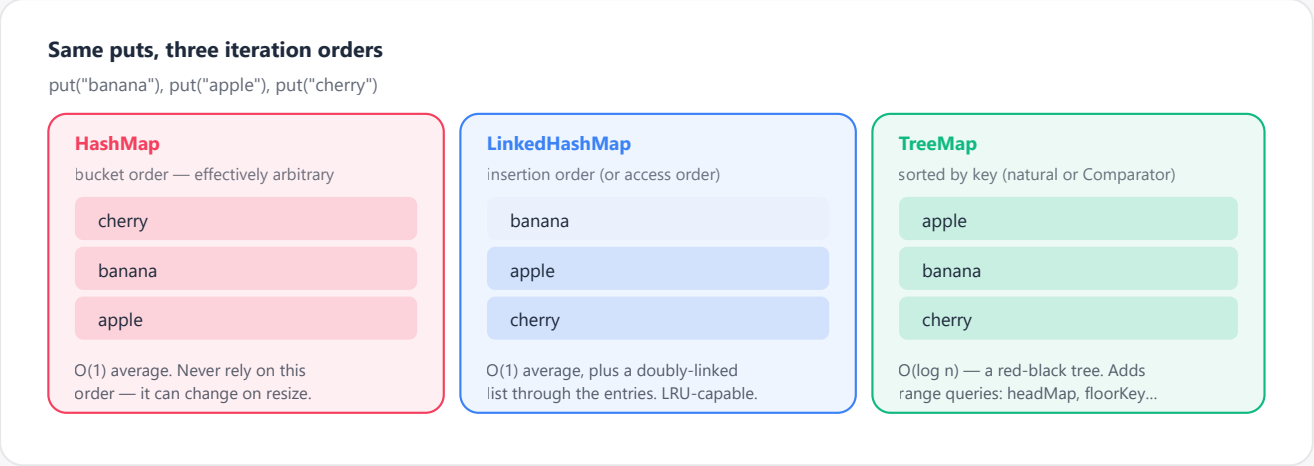
Bookmark  Mark done

## Why it matters

`HashMap`'s iteration order is arbitrary, and code that quietly depends on it is a bug waiting for a resize. Knowing the two ordered alternatives — and their costs — is what lets you answer "how would you build an LRU cache?" or "how do you find all entries between two keys?" without inventing something from scratch. Both questions are common, and both have a one-line answer if you know these classes.

## The concept, simply

Three maps, three orderings, three costs:




The same three puts, three different iteration orders. HashMap's is an artefact of hashing and may change on resize — never depend on it.

- `LinkedHashMap` extends `HashMap` and threads a doubly-linked list through the entries. Full O(1) hash performance, plus a predictable order — insertion order by default, or *access* order on request.

## How it works

| | `HashMap` | `LinkedHashMap` | `TreeMap` |
|---|---|---|---|
| Order | None (bucket order) | Insertion, or access | Sorted by key |
| get / put | O(1) average | O(1) average | O(log n) |
| Memory | Baseline | +2 references per entry | Tree nodes |
| null key | One allowed | One allowed | **No** — must be comparable |
| Extra API | — | `removeEldestEntry` | `firstKey`, `floorKey`, `head` |

`LinkedHashMap` **'s access-order mode** is the feature worth knowing by heart. Construct it with `accessOrder = true` and every `get` moves that entry to the end of the list. Combine that with the overridable `removeEldestEntry` hook and you have a complete LRU cache in about eight lines — which is a genuinely common interview exercise.

`TreeMap` **requires ordering**, either because the keys implement `Comparable` or because you supplied a `Comparator`. This has a consequence people miss: `TreeMap` **decides equality with** `compareTo`**, not** `equals`. If a comparator says two keys compare as 0, the map treats them as the same key regardless of what `equals` says. That is a powerful feature (case-insensitive maps) and a subtle bug source.

## Code examples

The LRU cache — a standard interview question with a standard answer:

JAVA Copy

```java
    public LruCache(int capacity) {
        super(16, 0.75f, true);          // ← true = ACCESS order, not ins
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;        // evict the least recently used
    }
}

LruCache<String, String> cache = new LruCache<>(3);
cache.put("a", "1"); cache.put("b", "2"); cache.put("c", "3");
cache.get("a");                          // "a" is now the most recent
cache.put("d", "4");                     // evicts "b" — the least recentl
System.out.println(cache.keySet());      // [c, a, d]

// Note: this is NOT thread-safe, and get() structurally modifies the map
// access-order mode — so even concurrent READS need synchronisation here.
// For a production cache, use Caffeine.
```

`TreeMap`'s navigation methods, which are the reason to choose it:

JAVA Copy

```java
TreeMap<Integer, String> scores = new TreeMap<>();
scores.put(70, "C"); scores.put(85, "B"); scores.put(95, "A");

scores.firstKey();            // 70   — lowest
scores.lastKey();             // 95   — highest
scores.floorKey(90);          // 85   — greatest key ≤ 90
scores.ceilingKey(90);        // 95   — smallest key ≥ 90
scores.higherKey(85);         // 95   — strictly greater
scores.headMap(85);           // {70=C}          — strictly below
scores.tailMap(85);           // {85=B, 95=A}    — from 85 up
scores.subMap(70, 90);        // {70=C, 85=B}    — a RANGE QUERY

scores.descendingMap();       // reversed view
```

```
// These are views, not copies — cheap, and they write through.
// Grade lookup in one line, instead of a chain of if/else:
String grade = scores.floorEntry(88).getValue();   // "B"
```

The `compareTo` -not- `equals` subtlety:

JAVA Copy

```
// A case-insensitive map, for free:
Map<String, Integer> ci = new TreeMap<>(String.CASE_INSENSITIVE_ORDER);
ci.put("Hello", 1);
ci.put("HELLO", 2);
System.out.println(ci.size());        // 1  — the comparator says they are
System.out.println(ci.get("hello"));  // 2

// The same mechanism as a bug — a comparator on ONE field silently
// deduplicates records that equals() considers different:
Map<Person, String> byAge = new TreeMap<>(Comparator.comparingInt(Person::
byAge.put(new Person("Asha", 30), "x");
byAge.put(new Person("Ben", 30), "y");
System.out.println(byAge.size());     // 1 — Ben overwrote Asha. Rarely in
```

## Under the hood

`LinkedHashMap` **is genuinely cheap.** It inherits the entire `HashMap` machinery and only adds `before` / `after` references to each node plus head and tail pointers on the map. Ordering costs two pointer writes per insertion, not a sort. That makes it a nearly free upgrade whenever deterministic iteration matters — for reproducible output, deterministic tests, or serialising configuration.

`TreeMap` **is a red-black tree**: a self-balancing binary search tree that guarantees the longest path is at most twice the shortest, so every operation is O(log n) worst case — no degenerate cases, unlike a plain BST fed sorted input. It is worth naming the structure and the guarantee; "it is sorted" is a much weaker answer.

**Where each actually gets used:** `LinkedHashMap` shows up wherever output must be reproducible (JSON field order, deterministic tests) and in caches. `TreeMap` shows up

every request, a `TreeMap` was the right structure.

## Common mistakes

- **Depending on `HashMap` iteration order.** It is arbitrary and can change on resize. Use `LinkedHashMap`.
- **Sorting a `HashMap`'s keys repeatedly.** If you need order every time, use a `TreeMap` and stop paying the sort.
- **Forgetting `TreeMap` uses `compareTo` for equality.** A comparator returning 0 merges entries `equals` would keep apart.
- **Putting a null key in a `TreeMap`.** Throws — there is nothing to compare.
- **Using an `LruCache` like the one above concurrently.** In access-order mode even `get` mutates the structure; it needs full synchronisation, or a real cache library.
- **Mutating a `TreeMap` key's sort field after insertion.** Same failure as a mutated hash key — the entry is stranded in the wrong position.

## Interview lens

**Q: "`HashMap` vs `LinkedHashMap` vs `TreeMap`?"**
A: `HashMap` is O(1) average with no ordering guarantee. `LinkedHashMap` is a `HashMap` with a doubly-linked list threaded through the entries, so it keeps insertion order (or access order) at O(1) and roughly two extra references per entry. `TreeMap` is a red-black tree sorted by key: O(log n), no nulls, but it gives range queries and navigation methods nothing else offers.

**Q: "How would you implement an LRU cache?"**
A: Extend `LinkedHashMap`, pass `accessOrder = true` to the three-argument constructor so every `get` moves the entry to the end, and override `removeEldestEntry` to return `size() > capacity`. That is the whole implementation. If asked to do it from scratch, it is a `HashMap` plus a doubly-

linked list — the map for O(1) lookup, the list for O(1) reordering and eviction, which is exactly what `LinkedHashMap` is.

**Q: "When would you use a `TreeMap`?"**

A: When the question is a range rather than a point. Pricing or grade tiers with `floorEntry`, time-series buckets with `subMap`, "the next event after time T" with `ceilingKey`. Also whenever I need to iterate in sorted order repeatedly — sorting a `HashMap`'s keys on every request is strictly worse.

**Q: "Does `TreeMap` use `equals`?"**

A: No — it uses `compareTo` or the supplied `Comparator`, and treats a result of 0 as "same key". That is how `new TreeMap<>(String.CASE_INSENSITIVE_ORDER)` gives a case-insensitive map for free, and also how a comparator on a single field can silently overwrite entries that `equals` would have kept distinct. The javadoc recommends the ordering be *consistent with equals* for exactly this reason.

**Q: "Can `TreeMap` hold a null key?"**

A: No — inserting one throws `NullPointerException`, because the map must compare the key and there is nothing to compare. Null *values* are fine. `HashMap` and `LinkedHashMap` each allow one null key.

**Q: "What ordering does `LinkedHashMap` use?"**

A: Insertion order by default — reinserting an existing key does not move it. With the three-argument constructor and `accessOrder = true`, it switches to access order and every `get` or `put` moves that entry to the end, which is what makes LRU eviction possible.

## Cheat-card

- `HashMap` O(1) no order · `LinkedHashMap` O(1) + order · `TreeMap` O(log n) + sorted.

- `TreeMap` = **red-black tree**, O(log n) worst case guaranteed.
- Navigation: `floorKey` `ceilingKey` `headMap` `tailMap` `subMap` — **views, not copies**.
- `TreeMap` equality is `compareTo`, **not** `equals`. No null keys.
- Never rely on `HashMap` order — it can change on resize.

## QUICK CHECK — no marks, just for you.

1. How do you build an LRU cache from the JDK's collections?

- ◯ Use an ArrayDeque and move entries to the front on access
- ◯ Use a TreeMap sorted by last access time
- ◯ Use a ConcurrentHashMap with a background eviction thread
- ◯ Extend LinkedHashMap with accessOrder=true in the three-arg constructor and override removeEldestEntry to return size() > capacity

Check answer

2. TreeMap decides whether two keys are the same using…

- ◯ compareTo or the supplied Comparator — a result of 0 means 'same key', regardless of what equals says
- ◯ equals and hashCode, like HashMap
- ◯ reference identity
- ◯ equals only

3. Which map would you choose to answer 'which pricing tier applies to a spend of 88?'

- ○ LinkedHashMap, relying on insertion order
- ○ TreeMap, using floorEntry(88) to get the greatest key not exceeding 88 in O(log n)
- ○ Any of them — the choice makes no difference
- ○ HashMap, iterating the keys and comparing

Check answer

‹ Previous
HashMap Internals

Next ›
Sets: HashSet, LinkedHashSet and TreeSet

## LEARN MORE

### ARTICLES & DOCS

- LinkedHashMap and removeEldestEntry — API docs
  docs.oracle.com
- TreeMap and NavigableMap — API docs
  docs.oracle.com

### VIDEOS



"linkedhashmap and treemap" — video explainers
YouTube



linkedhashmap and treemap interview questions